1. Fridge / Nested Shelves (Item Printer)

- **Scenario:** A fridge has shelves. Some shelves have smaller boxes inside. Each box contains items (like milk, eggs, veggies).
- **Task:** Print all items in the fridge no matter how deeply nested the boxes are.
- **Hint:**
  - Base case → item is not a box → print it
  - Recursive case → item is a box → call function on it

```
fridge = {
  "top_shelf": {
    "milk": 1,
    "butter": 2,
    "small_box": {
      "cheese_slices": 5,
      "chocolate": 3
    }
  },
  "middle_shelf": {
    "eggs": 12,
    "juice": 2
  },
  "bottom_shelf": {
    "vegetable_box": {
      "carrot": 6,
      "broccoli": 2,
      "leafy_box": {
        "spinach": 1,
        "lettuce": 1
      }
    },
    "water_bottle": 4
  }
}
```

2. Toy Boxes (Count All Toys)

- **Scenario:** You have boxes of toys. Some boxes have smaller boxes inside.
- **Task:** Count the total number of toys in all boxes.
- **Hint:**
  - Base case → item is a toy → count 1
  - Recursive case → item is a box → call function on it and sum

```
toy_boxes = {

    "Box A": {

        "Toy 1": "toy",

        "Box B": {

            "Toy 2": "toy",

            "Box C": {

                "Toy 3": "toy",

                "Toy 4": "toy"

            }

        },

        "Toy 5": "toy"

    }

}
```

3. Family Tree (Count Descendants)

- **Scenario:** Each person may have children, and each child may have their own children.
- **Task:** Count the total number of descendants for a given person.
- **Hint:**
  - Base case → person has no children → return 0
  - Recursive case → person has children → count each child + recursively count their descendants

```
family_tree = {

    "Grandparent": {

        "Parent A": {

            "Child 1": {},

            "Child 2": {}

        },

        "Parent B": {

            "Child 3": {},

            "Child 4": {

                "Grandchild 1": {}

            }

        }

    }

}
```

Folder Structure (Print All Files)

- **Scenario:** You have folders with files and subfolders.
- **Task:** Print the names of all files in a folder, including subfolders.
- **Hint:**
  - Base case → item is a file → print it
  - Recursive case → item is a folder → call function recursively

```
computer = {

    "file1.txt": "file",

    "Documents": {

        "resume.docx": "file",

        "Photos": {

            "pic1.jpg": "file",

            "pic2.jpg": "file"

        },

        "Notes": {

            "note1.txt": "file"

        }

    },

    "Music": {

        "song1.mp3": "file",

        "song2.mp3": "file"

    }

}
```

Nested List Sum

- **Scenario:** A list contains numbers or nested lists of numbers.
- **Task:** Calculate the total sum of all numbers in the nested list.
- **Example:** `[1, [2, 3], [4, [5, 6]]]` → sum = 21
- **Hint:**
  - Base case → element is a number → add it to sum
  - Recursive case → element is a list → call function on it
  -

. Nested Dictionary Key Search

- **Scenario:** You have a nested dictionary with unknown depth.
- **Task:** Find whether a specific key exists anywhere in the dictionary.
- **Hint:**
  - Base case → current level is empty → return False
  - Recursive case → key in dict → return True, otherwise call recursively on nested dictionaries
  -

Company Hierarchy / Employee Reporting

Scenario:

- A manager has employees, some of whom are managers themselves with their own team.
- You want to **count total employees under a manager**, including all levels of the hierarchy.

Data Concept: Nested dictionaries mapping manager → their direct reports.

- Base case → employee has no reports → return 0
- Recursive case → employee has reports → count them recursively

```
company = {
    "CEO": {
        "Manager A": {
            "Employee 1": {},
            "Employee 2": {}
        },
        "Manager B": {
            "Employee 3": {},
            "Manager C": {
                "Employee 4": {},
                "Employee 5": {}
            }
        }
    }
}
```

Recipe / Ingredients Breakdown

Scenario:

- A recipe contains ingredients. Some ingredients are **composite**, like “cake mix” which itself has flour, sugar, eggs.
- You want to **list all raw ingredients** needed for the recipe.

Data Concept: Dictionary: ingredient → quantity or sub-ingredients.

- Base case → raw ingredient → print or count
- Recursive case → composite ingredient → call recursively

```
recipe = {
    "Cake": {
        "Flour": "raw",
        "Sugar": "raw",
        "Eggs": "raw",
        "Frosting": {
            "Butter": "raw",
            "Sugar": "raw",
            "Vanilla": "raw"
        }
    }
}
```